


# Compiladores

## Aula 14: Tabela de símbolos e escopos

**Prof. Alex Torquato Souza Carneiro**
**alex.carneiro@impacta.edu.br**

# Agenda

- Modelagem
- Escopo
- Resolução de identificadores
- Implementações

# Modelagem

- Cada elemento do compilador tem uma atuação específica:
  - O scanner reconhece lexemas;
  - O parser revela a estrutura; e
  - A análise semântica resolve os nomes.




Identificadores deixam de ser apenas lexemas: cada uso é associado à declaração visível correta.

# Modelagem

- Sobre a tabela de símbolos, esta é construída incrementalmente durante a análise e consultada por etapas posteriores.
- Os campos concretos dependem da linguagem e das próximas fases do compilador.
- Livro-texto: lexema, tipo, endereço e demais atributos relevantes, reunidos em registros consultáveis.

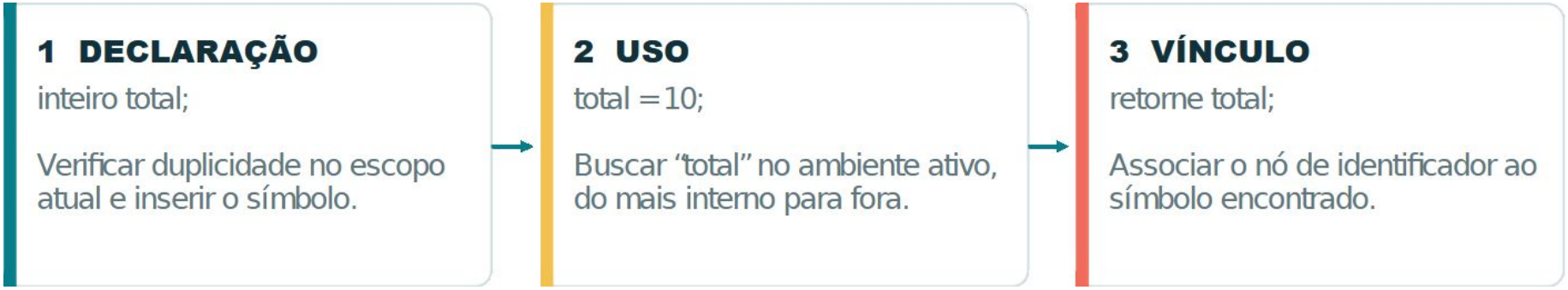
# Modelagem

- Exemplo de tabela de símbolos.

| CAMPO | EXEMPLO | PARA QUE SERVE |
|---|---|---|
| **nome** | `total` | identificar o lexema |
| **categoria** | variável / vetor / função / parâmetro | validar o tipo de uso |
| **tipo** | inteiro / real / lógico / retorno | análise de tipos e tradução |
| **escopo** | global / função / bloco | determinar visibilidade |
| **posição** | linha e coluna | diagnóstico preciso |
| **atributos** | tamanho / assinatura / deslocamento | informação específica |

# Escopo

- Toda declaração entra no escopo em que aparece. Cada uso procura uma declaração visível.




DECLARAÇÃO → ENTRADA NA TABELA → REFERÊNCIA NA AST

# Escopo

- É importante destacar que escopo não é a tabela.
  - escopo é uma região do programa;
  - tabela armazena declarações;
  - ambiente reúne tabelas visíveis.

**ESCOPO**

Parte do programa em que uma declaração pode ser usada. A regra é definida pela linguagem.

**TABELA**

Conjunto de entradas de um escopo: nome, tipo, categoria, posição e outros atributos.

**AMBIENTE**

Conjunto de tabelas acessíveis no ponto atual; pode ser representado por uma cadeia ou pilha.



# Resolução de identificadores

- No escopo léxico, o primeiro nome encontrado, do mais interno para o global, é o vínculo.




```
buscar(nome, atual):
    escopo = atual
    enquanto escopo != nulo:
        se nome em escopo.tabela:
            retornar escopo.tabela[nome]
        escopo = escopo.pai
    erro: identificador não declarado
```

**Custo intuitivo: proporcional à profundidade até o nome.**

# Resolução de identificadores

- Tomemos um identificador `x` que ocorre em dois blocos.
- Considere escopo léxico e permissão de sombreamento entre blocos, como no exemplo a seguir.

```
inteiro x;     // global
{
    real x;    // bloco interno
    x = 2.5;   // qual x?
}
x = 1;         // qual x?
```

**DENTRO DO BLOCO**

A busca encontra primeiro x : real. A declaração interna sombreia a global.

**DEPOIS DE "}"**

O lexema é igual; o símbolo associado ao uso depende do ponto do programa.

# Resolução de identificadores

- Para definir a estrutura do escopo atual, seguimos a seguinte regra:
  - Entrar em um bloco abre uma tabela;
  - Sair do bloco restaura o ambiente envolvente.
- Essa regra caracteriza uma pilha, onde novos escopos são inseridos quando ocorrem no código e removidos ao serem finalizados.



# Implementações

- Python e C podem representar a mesma ideia: a escolha afeta simplicidade, ownership e busca.

```python
# Python: uma tabela por escopo
class Escopo:
    def __init__(self, pai=None):
        self.pai = pai
        self.simbolos = {}

    def buscar(self, nome):
        if nome in self.simbolos:
            return self.simbolos[nome]
        return self.pai.buscar(nome) if self.pai else
None
```

```c
/* C: cadeia de escopos */
typedef struct Simbolo {
  char *nome;
  int categoria, tipo, linha, coluna;
  struct Simbolo *prox;
} Simbolo;

typedef struct Escopo {
  Simbolo *tabela;
  struct Escopo *pai;
} Escopo;
```

A operação declarar deve rejeitar duplicidade no escopo atual, não necessariamente no pai.

# Implementações

- Uma declaração interna pode sombrear uma externa; duas declarações iguais no mesmo escopo podem ser erro.

```
declarar(simbolo, atual):
    se simbolo.nome em atual.tabela:
        erro “declaração duplicada”
    senão:
        atual.tabela[simbolo.nome] = simbolo
```

**MESMO ESCOPO**

inteiro x;
real x; → duplicidade, conforme a regra adotada.

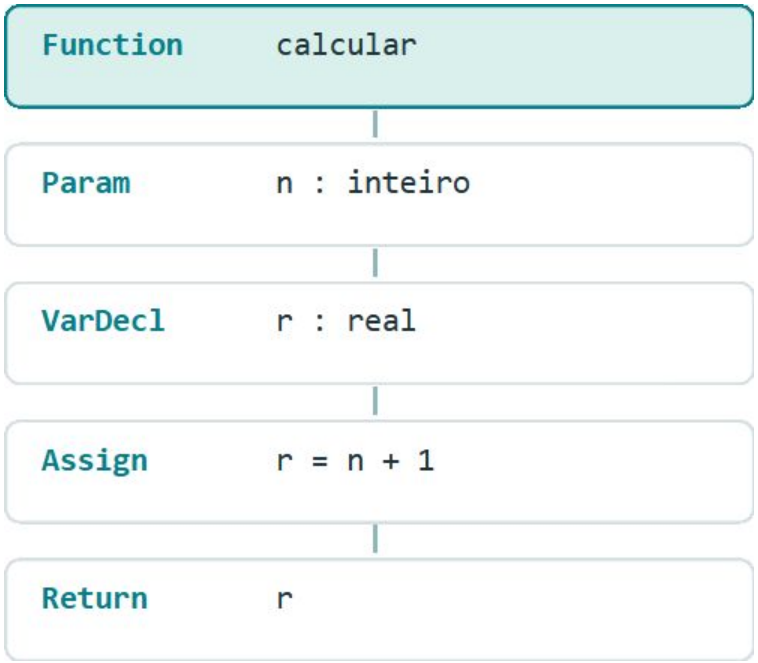
**ESCOPO ANINHADO**

inteiro x; {real x; }
→ nomes distintos; o interno sombreia o externo.

A política exata (por exemplo, permitir ou não sombreamento) pertence à especificação da linguagem.

# Implementações

- Na visita semântica, declarações criam símbolos; identificadores recebem referências aos símbolos.



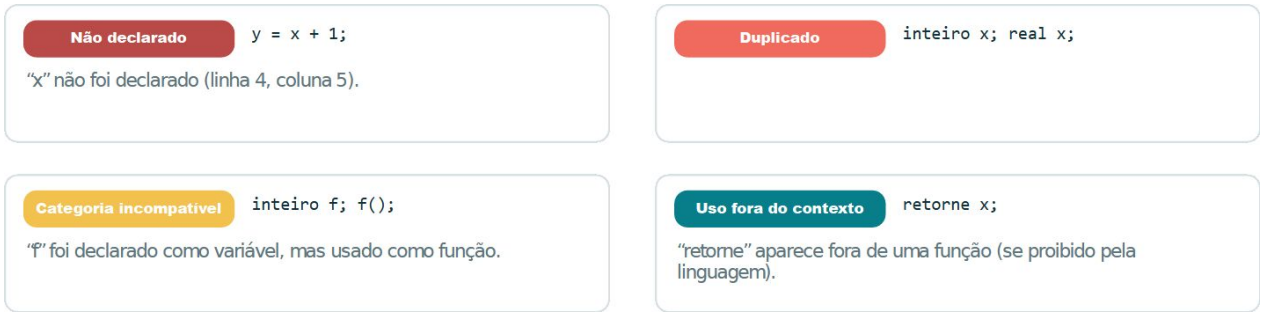

```
visitar(VarDecl d):
    símbolo = criar(d)
    atual.inserir(símbolo)

visitar(Id i):
    i.simbolo = ambiente.buscar(i.nome)
    se i.simbolo == nulo: erro
```

Resultado: cada ocorrência de “n” e “r” aponta para sua declaração.

# Implementações

- A tabela ajuda a explicar o que está errado.
- Uma mensagem de erro apropriada inclui:
  - categoria do erro,
  - lexema e
  - localização;
  - quando útil, aponta a declaração conflitante.




Posição do uso + posição da declaração conflitante = diagnóstico mais acionável.

# Atividade proposta

- Use uma pilha de tabelas e registre a origem de cada identificador.

```
inteiro x;
inteiro principal(inteiro n) {
    real y;
    {
        lógico x;
        x = verdadeiro;
        y = 2.5;
    }
    x = 3;
    retorne n;
}
```

1. Quais símbolos existem em cada escopo?
2. A que declaração se liga cada uso de `x`?
3. O que muda quando o bloco interno fecha?
4. Que erros surgem se o último `n` não foi declarado como parâmetro?

**Entrega: diagrama + tabela de símbolos + justificativa.**